

aLOTofImaginArches: interactive graphical statics of masonry arches in the browser

G. Castellazzi

Department of Civil, Chemical, Environmental and Materials Engineering (DICAM), University of Bologna, Viale Risorgimento 2, 40136 Bologna, Italy

`giovanni.castellazzi@unibo.it`

Abstract

The equilibrium of a masonry arch is not a single state but a three-parameter family of them, and that multiplicity is the hardest idea to convey when the subject is taught. *aLOTofImaginArches* makes it manipulable. A student loads a photograph of a real arch, traces its intrados and extrados, subdivides the ring into voussoirs, sets a scale, applies loads, and then moves the three degrees of freedom of the line of thrust while the software reports, joint by joint, whether the line stays within the masonry: Heyman's safe theorem, made operable. The software is a rewrite of an established MATLAB application as dependency-free ES modules that run directly from a static host, so that no licence and no installation stand between a student and the construction. The graphical idiom of the original is preserved deliberately. Twenty-seven legacy examples are carried over and fifteen of them, which store a complete solution, are reproduced to machine precision.

Keywords: masonry arches, line of thrust, graphical statics, limit analysis, engineering education, web application

1. Motivation and significance

A masonry arch stands not because it is in *the* state of equilibrium but because it is in *one* of infinitely many. Under Heyman's assumptions — no tensile strength, unlimited compressive strength, no sliding — the safe theorem states that if any line of thrust in equilibrium with the applied loads can be found lying wholly within the masonry, the structure is safe [1, 2, 3]. For a two-dimensional arch that line has three degrees of freedom, the ∞^3 of the classical literature [4, 5].

Students meet this in a lecture and, in the author's experience of teaching it, do not believe it. The obstacle is not the algebra of the funicular polygon, which is elementary, but the multiplicity: a single worked example produces a single drawing, and a single drawing looks like a single answer. What convinces is moving the solution and watching it remain a solution — and then watching it stop being an *admissible* one as it leaves the ring.

Interactive equilibrium tools have a distinguished record in exactly this role [6, 7, 8], and the present software follows that line for the particular case of the two-dimensional arch, with an emphasis on the historical construction rather than on generality. Its predecessor,

aLOTofImaginArches, is a MATLAB App that has been used for several years in the master's courses *Mechanics of Historical Masonry Structures* and *Historical Masonry and Wooden Structures* at the University of Bologna. Two things limited it. It required a MATLAB licence, which excludes students working on their own machines and anyone outside a subscribing institution; and its source, packaged in a single binary `.mlapp` container, was not something a student could read.

This paper describes a rewrite as a browser application: plain ES modules with no dependencies and no build step, served as static files, so that the source a student reads is exactly the source that runs. Three commitments shaped it.

1. **The graphics of the original are preserved deliberately.** The boxed axes with inward ticks, the pale grid, the translucent force bands and the constant-size arrowheads of the MATLAB figure are reproduced on the HTML canvas. Continuity of appearance is not nostalgia: the course notes, the slides and the existing tutorials all show that idiom, and a student switching between them should not have to re-learn what a picture means.

Table 1: Code metadata

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v1.0.0
C2	Permanent link to code/repository used for this code version	https://github.com/gcastellazzi/aLOTofImaginArches
C3	Permanent link to Reproducible Capsule	The application itself is the capsule: it is served as static files and runs, unmodified, at https://gcastellazzi.github.io/aLOTofImaginArches/app/
C4	Legal Code License	MIT License
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	JavaScript (ES2022 modules), HTML5 Canvas, CSS; Python 3 for the legacy-format converter; GitHub Pages
C7	Compilation requirements, operating environments, dependencies	None. Any modern browser runs the application as served; there is no build step and no third-party runtime dependency. Node.js ≥ 18 is needed only to run the test suite; Python 3 with NumPy and SciPy only to re-run the converter.
C8	Link to developer documentation/manual	https://gcastellazzi.github.io/aLOTofImaginArches/ and the README.md files under docs/app/
C9	Support email for questions	giovanni.castellazzi@unibo.it

2. **The mechanics is separated from the drawing and is tested.** The construction lives in seven small modules with no reference to the DOM, so it can be, and is, exercised by a test suite from the command line.
3. **The legacy examples are carried over, and checked.** Every stored example of the MATLAB version was converted, and the conversion was verified field by field rather than assumed.

2. Software description

2.1. Software architecture

The application is a static web page. There is no server, no bundler and no package to install: the browser loads `index.html`, which imports ES modules directly. Table 2 lists them.

The separation is strict in one direction: `core` knows nothing of the canvas, and `render` knows nothing of arches. It is what allows the mechanics to be checked without a browser. The suite comprises 72 tests run by the Node test runner (`node -test`), covering the geometry, the funicular construction, the reproduction of stored examples, tracing and block generation, scale and units, admissibility, and the save format.

A separate Python tool, `tools/mat2json.py`, converts the MATLAB `.mat` examples to JSON and extracts the background images. It carries a `-check` mode that re-reads every converted file and compares it field by field with the original, so the conversion is verified rather than trusted.

Table 2: Modules, with their size in lines. The `core` modules contain no reference to the browser and are the ones the test suite drives.

Layer	Module	Lines
core	<code>geometry.js</code>	117
	<code>blocks.js</code>	138
	<code>statics.js</code>	294
	<code>trace.js</code>	192
	<code>units.js</code>	164
	<code>model.js</code>	150
	<code>persist.js</code>	169
render	<code>axes.js</code>	325
	<code>draw.js</code>	246
—	<code>app.js</code>	1036
Total		2831

2.2. Software functionalities

Stored examples. Twenty-eight example files are shipped. One is an empty template; of the remaining twenty-seven, fifteen carry a complete stored solution and are recomputed from their geometry alone, six were saved before a solution was computed, and six are internally inconsistent — the stored solution does not correspond to the stored geometry. The software detects the last group and says so, rather than drawing a solution it cannot justify (Section 3.1).

Tracing. A student loads any image and clicks along the intrados and the extrados. The two curves are re-sampled by arc length and joined into n voussoirs with $n + 1$ radial joints. The trace is checked before it is used: too few points, coincident curves, and — the case that matters — curves that cross are all reported. A crossing

Table 3: The three parameters that fix the line of thrust.

Parameter	Meaning
Pole abscissa	the horizontal thrust H
Start fraction s	where the line leaves its springing joint; 0 at the intrados, 1 at the extrados
Pole ordinate	how the total weight divides between the two vertical reactions

is detected by the *sign* of the signed area of the resulting blocks, not by their size, because an inverted block can be as large as a sound one.

Scale and units. Until a scale is set, every quantity is in pixels. The student picks two points on the image and states the real distance between them; lengths then scale by k , areas by k^2 , and weights by k^2 or k^3 according to whether the out-of-plane thickness was given physically or in pixels. Three unit systems are offered (SI, N–mm, kgf–cm), matching the original. Span and rise are reported after scaling, because they are the quickest check that the reference distance was entered correctly.

Loads. Point loads are placed by clicking. A load and a voussoir weight are treated identically by the construction — both are stations on the load line — so applied forces are merged into the same ordered sequence as zero-area blocks.

The three degrees of freedom. The pole of the force polygon has two coordinates and the line has one free end, giving the three parameters of Table 3, each on its own slider. The far end is not a fourth parameter: the last ray is carried on until it meets the other joint, and where it lands is reported.

Admissibility. For each joint the software computes where the line crosses it, as a fraction s of the joint. The verdict names the number of joints violated, the worst one and on which face, or — when the line fits — how much room is left. Choosing the crossing correctly is less trivial than it appears: a joint near a springing is almost horizontal, and its infinite line meets segments of the thrust line far away that have nothing to do with it, so the implementation prefers the crossing that lies *within* the joint.

Hooke’s cable. The same polygon reflected about the chord joining the two springings, so that the cable hangs from the points the arch springs from whatever their relative height, and *ut pendet continuum flexile, sic stabit contiguum rigidum inversum* [9] can be shown rather

than quoted. The rays of the force polygon are drawn dashed and lettered after Bow, with the same letter on the parallel segment of the thrust line, so that a force can be carried from one drawing to the other by eye.

Saving. The whole session — traced curves, blocks, joints, weights, loads, scale, unit system and the three parameters — is written to a single JSON file and read back. Everything derivable is deliberately omitted and recomputed on opening, so a file saved by one version stays readable when the mechanics is corrected. A file is validated before it is trusted and refused with a diagnostic if it is inconsistent, rather than half-loaded.

3. Illustrative examples

3.1. Reproducing Poleni

Figure 1 shows the software working over Poleni’s own plate of 1748 for the dome of St Peter’s [10], the drawing in which the hanging-chain analogy was first put to a real structure. The 56 voussoirs and their weights come from the stored example; the force polygon and the line of thrust are recomputed from that geometry by the JavaScript implementation. The largest discrepancy between the recomputed line and the one MATLAB saved is 8.0×10^{-16} relative — machine precision. Across all fifteen examples that carry a complete solution, the worst relative discrepancy is 8.5×10^{-16} for the force polygon and 3.9×10^{-15} for the line of thrust.

That agreement is worth stating precisely because the remaining examples do *not* agree, and for identifiable reasons: in some the stored force polygon was drawn with weights that no longer match the stored blocks; in others, applied loads were merged into the sequence but the loads themselves were never written to the file. This is a property of the original save format, not of either implementation, and it was found only because the port was checked against the original rather than eyeballed. The software now reports these files as not recomputable and displays the stored solution with a warning.

3.2. The minimum thickness of a semicircular arch

The second example is the classical one, and it exposed a defect worth recording. For a semicircular ring of uniform thickness the least t/r_i at which an admissible line exists is about 0.108 [3, 11].

An early version of this software pinned both ends of the line at the mid-points of the end joints, leaving only the thrust free. Measured by bisection on a ring of 16 voussoirs, that one-parameter family first admits a line

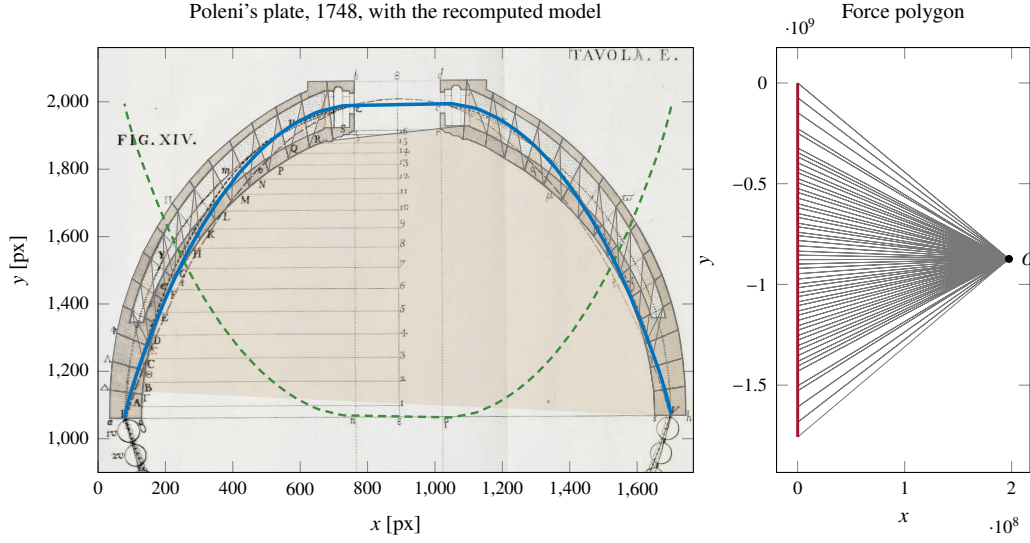


Figure 1: Poleni's plate of 1748 with the model recomputed by the software: 56 voussoirs (grey), the line of thrust (blue) and the inverted Hooke cable (green), with the force polygon on the right. The line is recomputed from the geometry alone and agrees with the solution stored by the MATLAB application to 8.0×10^{-16} relative.

Table 4: Least t/r_i admitting a line of thrust on a semicircular ring, by bisection, against the number of voussoirs n . Heyman's value for a continuous ring is 0.108.

n	ends pinned	ends free
8	0.1789	0.1116
12	0.1894	0.1118
16	0.1984	0.1155
24	0.2070	0.1181
32	0.2061	0.1179
48	0.2085	0.1199
64	0.2093	0.1202

at $t/r_i = 0.198$ — nearly twice Heyman's value. The tool was therefore condemning arches that stand.

Two things were wrong. The ends were pinned, discarding two degrees of freedom; and, less obviously, the funicular construction inherited from the original stops on the *vertical* through the far springing rather than reaching the far joint at all, so asking where the line crossed that joint returned the intersection of the joint's infinite line with an unrelated segment. Freeing the start along its joint and carrying the last ray on to the far joint restores the full family. Table 4 gives the measured result.

With the ends free the figure settles at about 0.12 and approaches Heyman's 0.108 from above; the residual gap is discretisation, a funicular polygon of n segments being able to cut corners that a continuous curve cannot,

and it narrows as n falls rather than rises for exactly that reason. More telling than the number is that the search, given no guidance, recovers the textbook limit state on its own: the critical line leaves one springing at the *extrados*, arrives at the other at the *extrados*, and requires a horizontal thrust of about one fifth of the total weight.

Figure 2 shows three states of the same ring at $t/r_i = 0.15$, a thickness for which the pinned family admits nothing at all. Figure 3 gives the admissible range of thrust as a function of thickness for both families: the safe theorem, drawn.

4. Impact

The immediate impact is on teaching. The software is used in the master's course *Mechanics of Historical Masonry Structures* at the University of Bologna, alongside a set of lecture slides, handouts and exercises derived from [12]. Removing the licence requirement changes what can be asked of students: an exercise can now be set as *photograph an arch in this city, trace it, and find an admissible line*, and every student can do it on their own machine, at home, without an installation. The saved session is a single small JSON file, which makes such an exercise collectable and markable.

A second effect is on reproducibility. Because the port was verified against the original rather than reimplemented from the paper description, it produced a usable audit of a decade of stored examples, and identified

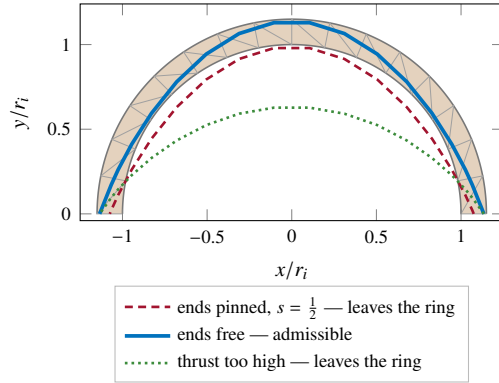


Figure 2: Three equilibrium states of the same semicircular ring, $t/r_i = 0.15$, $n = 16$. All three are in equilibrium with the weights; only the middle one is admissible. The symmetric line through the joint mid-points (dashed) is the best the pinned construction can offer at that thrust, and it leaves the ring.

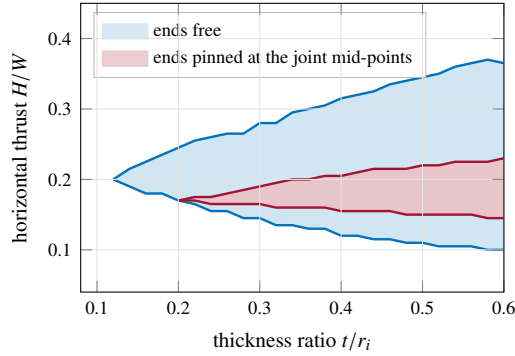


Figure 3: Range of horizontal thrust, as a fraction of the total weight, for which an admissible line exists, against the thickness ratio. Freeing the ends both lowers the thickness at which the band opens and widens it everywhere.

six that cannot be reproduced from what they contain. A researcher relying on those files would have had no warning. The general point — that a rewrite is an opportunity to audit the data of the original, and that the audit should be automated and kept — applies well beyond this software.

Third, the code is small enough to be read. Graphical statics is a subject in which the construction is simple and the bookkeeping is not, and a 300-line module in which `forcePolygon` and `funicular` sit next to each other with the classical terms as their variable names is itself a teaching artefact.

The scope is deliberately narrow, and the limitations follow from Heyman’s assumptions rather than from the implementation: the analysis is two-dimensional, blocks are rigid, sliding is not checked, and the finite

subdivision means the computed line is a polygon inscribed in the true funicular curve. None of these are hidden from the user; the discretisation, in particular, is a slider.

5. Conclusions

aLOTofImaginArches makes the three-parameter family of equilibrium states of a masonry arch something a student can take hold of and move. The rewrite from MATLAB to dependency-free browser modules removes the licence and the installation, preserves the graphical language of the original, and — by separating the mechanics from the drawing — makes the construction testable from the command line. The legacy examples are reproduced to machine precision where they can be reproduced at all, and the software says so when they cannot.

Freeing the two end positions of the line, which the original construction had fixed, brought the computed minimum thickness of a semicircular ring from $t/r_i = 0.198$ to 0.115 against Heyman’s 0.108 , and let the software recover the classical limit state without being told what to look for. Work in progress extends the same treatment to arches on spreading supports and to the sliding condition that Heyman’s third assumption sets aside.

Acknowledgements

The MATLAB predecessor was developed for, and improved by, successive cohorts of students of the courses *Mechanics of Historical Masonry Structures* and *Historical Masonry and Wooden Structures* at the University of Bologna.

References

- [1] A. Kooharian, Limit analysis of voussoir (segmental) and concrete arches, *Journal of the American Concrete Institute* 24 (4) (1952) 317–328.
- [2] J. Heyman, The stone skeleton, *International Journal of Solids and Structures* 2 (2) (1966) 249–279.
- [3] J. Heyman, *The Masonry Arch*, Ellis Horwood, Chichester, 1982.
- [4] J. Heyman, *The Stone Skeleton: Structural Engineering of Masonry Architecture*, Cambridge University Press, Cambridge, 1995.

- [5] S. Huerta, Mechanics of masonry vaults: The equilibrium approach, in: P. B. Lourenço, P. Roca (Eds.), *Historical Constructions 2001*, Guimarães, 2001, pp. 47–69.
- [6] D. O’Dwyer, Funicular analysis of masonry vaults, *Computers & Structures* 73 (1–5) (1999) 187–197.
- [7] P. Block, T. Ciblac, J. Ochsendorf, Real-time limit analysis of vaulted masonry buildings, *Computers & Structures* 84 (29–30) (2006) 1841–1852.
- [8] P. Block, J. Ochsendorf, Thrust network analysis: A new methodology for three-dimensional equilibrium, *Journal of the International Association for Shell and Spatial Structures* 48 (3) (2007) 167–173.
- [9] R. Hooke, *A Description of Helioscopes, and Some Other Instruments*, John Martyn, London, 1676.
- [10] G. Poleni, *Memorie istoriche della gran cupola del Tempio Vaticano*, Stamperia del Seminario, Padova, 1748.
- [11] S. Huerta, Galileo was wrong: The geometrical design of masonry arches, *Nexus Network Journal* 8 (2) (2006) 25–52.
- [12] G. Castellazzi, A. Taliercio, *Principles and Analysis of Historical Masonry Structures*, Cambridge Scholars Publishing, Newcastle upon Tyne, 2025.